

PREFIX SUM (2D)

```
vector<vector<long long>> pre(n+1, vector<long long>(m+1, 0));
for (int i=0;i<n;i++) for (int j=0;j<m;j++)
    pre[i+1][j+1] = grid[i][j] + pre[i][j+1] + pre[i+1][j] - pre[i][j];
long long rectSum = pre[r2+1][c2+1]-pre[r1][c2+1]-pre[r2+1][c1]+pre[r1][c1];
// (r1,c1)-(r2,c2)
```

Demo: answer Q rectangle-sum queries in O(1) each

```
while (q--) { cin>>r1>>c1>>r2>>c2;
    cout << pre[r2+1][c2+1]-pre[r1][c2+1]-pre[r2+1][c1]+pre[r1][c1] << "\n"; }
```

DIFFERENCE ARRAY

```
// size n+1 - the extra slot at r+1 cancels the update out
vector<long long> diff(n+1, 0);          diff[1]+=x; diff[r+1]-=x; // add x to [1,r]
long long cur=0; for(int i=0;i<n;i++){ cur+=diff[i]; a[i]=cur; } // reconstruct
```

Demo: apply Q range updates, then output final array

```
while (q--) {
    cin>>l>>r>>x; diff[l]+=x; diff[r+1]-=x;
}
long long cur=0; for(int i=0;i<n;i++){ cur+=diff[i]; cout<<cur<<" "; }
```

BINARY SEARCH (basic)

```
int l=0, r=n-1, ans=-1;
while (l <= r) {
    int mid = l + (r-l)/2;
    if (a[mid]==target) { ans=mid; break; }
    else if (a[mid]<target) l=mid+1;
    else r=mid-1;
}
binary_search(a.begin(), a.end(), target); // built-in, returns true/false only
```

LOWER BOUND / UPPER BOUND

```
lower_bound(a.begin(),a.end(),x); // first elem >= x
upper_bound(a.begin(),a.end(),x); // first elem > x
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end()) cout << *it; // direct iterator use, dereference for value
int idx = lower_bound(a.begin(), a.end(), x) - a.begin(); // convert to index
bool exists = (idx < n && a[idx] == x); // check existence
int cnt = upper_bound(a.begin(),a.end(),x) - lower_bound(a.begin(),a.end(),x);
// count of x
```

Demo: find the smallest element >= x in a sorted array

```
auto it = lower_bound(a.begin(), a.end(), x);
if (it != a.end()) cout << *it; else cout << "no such element";
```

===== MULTIDIMENSIONAL LOWER BOUND (on vector of pairs, sorted) =====

```
lower_bound(vp.begin(), vp.end(), make_pair(x, INT_MIN));
// first pair with .first >= x
```

// Demo: count how many elements in sorted array are <= x

```
int cntLE = upper_bound(a.begin(), a.end(), x) - a.begin();
```

// Demo: count how many elements in sorted array are in range [lo, hi]

```
int cntRange = upper_bound(a.begin(),a.end(),hi) - lower_bound(a.begin(),a.end(),lo);
```